

JARVIS X — MASTER BLUEPRINT v4

Agentic OS Update, Reconciliation & Self-Maintenance Charter

Prepared 9 September 2026 · Base commit 8643e6e · Machine: Asus Chromebook (i5-1135G7, 14 GB RAM, Crostini)

0. What this document is

This is a **reconciliation and update**, not a new plan written from scratch. It takes every document you uploaded — the September 3 evidence chain (AS_BUILT.md, MASTER_PLAN_v3.md, the repo triage, the rebuild runbook) and the August 28 status report and v2.0 “Ascension” plan — and produces one current, internally consistent blueprint you can act on.

Precedence rule used throughout: where two documents disagree, the one with cited evidence (a test name, a command output, a file path) wins over the one with a narrative claim. MASTER_PLAN_v3.md (Sept 3) is the newest evidence-based document and is treated as canonical current state. The Aug 28 status report and the v2.0 Ascension plan are folded in as *inputs*, not as overriding sources — see §2 for exactly where they conflict and how each conflict was resolved.

This document does four things: 1. States one current, evidence-backed picture of what exists (§1–2). 2. Flags a scope-discipline tension this batch of files reintroduces, and asks you to rule on it rather than silently deciding for you (§3). 3. Proposes the self-maintaining “Agentic OS” loop — how Jarvis X checks and repairs its own memory, code, and status going forward (§4). 4. Gives a concrete deployment path for the Chromebook, local and “online” (§5), and the next tasks in priority order (§6).

1. Ground truth: hardware and environment

Settled, not open. MASTER_PLAN_v3.md resolved the four-machine confusion from the runbook by tracing the 7.7 GB figure to a different, non-dev machine (a Lenovo Y50-70 spec sheet that doesn’t apply here).

Property	Value
Machine	Asus Chromebook
CPU	Intel i5-1135G7, 8 vCPU, AVX-512
RAM	14 GB, no swap

Property	Value
GPU	None
OS	ChromeOS + Crostini (Debian 12, kernel 6.6.119)
Ollama	0.32.9, with qwen2.5:7b and qwen2.5:3b pulled
Hostname	penguin

Still open: this profile is corroborated across two documents but was never captured fresh from your live terminal in this session. Session 1 of the runbook (\$6, one 30-minute command block) produces ~/PROFILE.md and closes this permanently. Recommended as the first task below.

2. Current verified state (supersedes every earlier completion percentage)

2.1 Why there were four different numbers, and why none of them is “the” number

Figure	What it actually measured	Verdict
91% (22/24)	scripts/status.sh milestones — 17 of 24 are bare file-existence checks	Measures inventory, not function
95% “production ready”	Narrative claim in JARVIS v2.pdf, Aug 12	No cited evidence found
75% (Aug 28 status report)	Weighted judgment across 7 stated goals, largely accurate for what it covers (voice/language)	Superseded by the Sept 3 evidence chain for anything outside voice
62% (Sept 3, MASTER_PLAN_v3.md)	Runbook’s phase weights (Planning/Setup/Build/Test /Delivery) scored against AS_BUILT.md’s cited evidence	Current canonical figure

The 62% and 75% figures aren’t actually contradictory once you see what each measures: the Aug 28 report is a close-up on the voice/language subsystem (where real regressions were found and fixed that week), while the Sept 3 figure is a whole-project rollup that weights untested and stubbed components down. Both can be true at once. Treat **62% as the whole-system number** and the Aug 28 fixes as now folded into “Build” and “Setup” in that rollup.

2.2 Verified working (cited evidence, not narrative)

Component	Evidence
Path jail (code/exec.js)	Fixed a jail-root bug this cycle; test-exec.js 2/2
Shell execution (code/shell.js)	Fixed a jail-escape bug (cwd not set on spawnSync); reproduced and closed
Action validation	test-validate.js 23/23
Model gateway (breaker, budget, store, telemetry, tier policy)	47/47 across 8 test files
Data layer + provider registry	5/5; market, crypto, energy, news providers registered
Scheduler	8/8
i18n (EN/AR) + accessibility	3/3 and 5/5 (after declaring the missing i18next dependency)
Voice routing table	4/4 routing cases
jj CLI	Was a hard syntax error (shell comments in a .js file); fixed, now runs
Arabic text generation	SILMA-9B, correct Egyptian dialect (Aug 28 regression test)
Voice synthesis, short + long form	Fixed a romanisation bug that was silently converting Arabic to Latin script before it reached the TTS engine; both forms verified with real audio output
API endpoints	All return HTTP 200 after removing a duplicate catch-all route that was shadowing two of them

2.3 Exists but unverified, stubbed, or genuinely unresolved

Item	Status
Ollama vision, 3 voice paths, Kokoro TTS, live-data test	Unmeasured , not known-broken — blocked by missing local models/deps in the test environment, not by code defects. Highest-value single task: run these on the real Chromebook.
jj status	Stub — prints a hardcoded “ready” message, checks nothing
test-guard.js, test-shell.js	Not real tests — zero assertions, always pass
JX_NET network-test gating	Declared in package.json, read nowhere — dead flag
.deb installer	Ships an empty /opt/jarvis-x; the launcher has nothing to run

Item	Status
Electron desktop shell	3 files only (<code>main.js</code> + package files), no verified build
<code>code/paper-trading.js</code> + <code>config/trading.json</code>	Exists, wired into nothing, no test — see §3, this needs your ruling, not a technical fix
E2E testing	One script, not a suite

2.4 The voice-stack naming conflict (flagging, not resolving)

Your documents name **three different TTS engines** for the same role at different points: Piper (runbook, memory), Kokoro-82M (your stated current choice, already downloaded), and a “Chatterbox Egyptian worker” / SILMA-9B pairing (Aug 28 status report, described as the one generating the measured audio). These are not compatible descriptions of one voice stack. Given Kokoro-82M is what you’ve actually confirmed downloading and adopting most recently, **this blueprint treats Kokoro-82M as current** and flags the Chatterbox/SILMA description as either an earlier experiment or a different machine’s config. **Needs your confirmation** — see the open-decisions table in §6.

3. Scope-discipline flag — please rule on this before build work resumes

You previously did the work of consolidating around twelve competing plans into one canonical, disciplined scope, with an explicit cut list: **no trading/market intelligence, no daily content pipeline, no multi-agent council, no patent claims, no self-mutating agent, no 6-agent orchestration.**

Two of the documents in this batch reintroduce exactly those items:

Reintroduced item	Where	Current build state
Market Intelligence / paper trading	JARVIS_X_v2_Ascension_Plan.pdf §7; also live as <code>code/paper-trading.js</code>	Already exists in the repo, unwired, no test. <code>MASTER_PLAN_v3.md</code> found it does not technically violate <code>CONSTITUTION.md</code> as written (which forbids only <i>real-money</i> trading) — but it does violate the cut list you set for yourself, which is a stricter, separate rule.
Content/monetization engine	Ascension Plan §6	Not built
Quantum “AI” module	<code>MASTER_PLAN_v3.md</code> §4–6	Built as an isolated, gated research track (<code>quantum/</code>), deliberately kept out of the

Reintroduced item	Where	Current build state
		request path. Measured and found to lose to a zero-parameter keyword matcher on every axis — the module’s own conclusion is “don’t ship this,” which happens to agree with your cut list by coincidence.
Universal multi-provider “Council” framing	Ascension Plan §3	This one is largely just a naming/framing change over the router you already have (Ollama + Gemini + Groq + OpenRouter) — likely fine to keep, not real scope creep.

This blueprint does not silently re-adopt the cut items. It keeps the Market Intelligence and Content Engine modules **parked, undocumented as active scope**, and keeps the quantum track exactly where MASTER_PLAN_v3.md already put it — gated, isolated, not in the request path, promoted only if it clears four measured gates that it currently fails. If you want any of these back in active scope, say so explicitly and this document gets a short addendum; otherwise the next section proceeds as if the original cut list still holds.

4. The Agentic OS layer — self-maintenance and continuous improvement

This is the part of your request that’s new: turning Jarvis X into a system that checks and repairs itself, not just a system you debug by hand each session.

4.1 Two loops, borrowed from the bitemporal-memory pattern you were just reading

The memory-repair architecture you reviewed (arrival path + sweep path converging on one writer) generalizes directly to *project health*, not just to conversational facts:

	Arrival path (event-driven)	Sweep path (time-driven)
In the memory guide	User says something → extract → classify → repair	Timer fires → detect drift nobody reported → propose → repair
Applied to Jarvis X itself	You (or Claude Code) make a change → tests run → AS_BUILT.md updates	A scheduled <code>jj status --deep</code> run finds rot nobody reported: a dependency that silently stopped being used, a

Arrival path (event-driven)	Sweep path (time-driven)
	test that quietly stopped asserting anything, a doc that drifted from the code

The same asymmetry applies: a broken build is easy to notice — CI turns red. A **stale-but-passing** check (like the 17 file-existence “milestones” that stayed green through an actual jail-escape bug) is the dangerous case, because nothing signals it. That is precisely what the Sept 3 triage found, and it’s worth building a permanent defense against rather than re-discovering it every few weeks.

4.2 Concrete self-maintenance mechanism to build

1. **A repairs-style audit table for the repo itself**, not just for memory — every automated fix Claude Code proposes gets one append-only row: what changed, why, evidence, confidence, who approved it. You already have the human-approval gate in your operating rules; this just gives it a permanent, queryable log instead of scattered session summaries.
2. **A weekly (not per-session) sweep**, run on a schedule via the existing scheduler component (already 8/8 tested): re-run the JS suite, re-run model-gateway tests, check for tests with zero assertions (the `test-guard.js / test-shell.js` failure mode), and diff `AS_BUILT.md`’s claims against fresh command output. Anything that drifted gets parked for your review — this reuses the gate/inbox pattern from the memory guide almost exactly.
3. **Confidence-gated auto-apply, never autonomous merge**. Per your standing rule (Claude Code as reviewed diff proposer only): routine, low-risk fixes (a missing dependency, a stale version pin) can be proposed with a diff and auto-flagged high-confidence; anything that deletes, disables, or changes the constitution always parks for your explicit approval — mirroring the memory guide’s two-threshold routing (0.80 to retire something, 0.60 to only add something).
4. **One evidence file, always current: `AS_BUILT.md` stays the single source of truth**. Every status document that isn’t `AS_BUILT.md` gets a one-line “superseded by `AS_BUILT.md`, <date>” header the moment a newer sweep runs — this is already your own stated rule from the runbook; the sweep just makes it automatic instead of manual.

This gives you “self-maintaining” in the honest sense your own hardware supports: cheap, SQL-and-file-read detection running on a timer, human-gated repair, full audit trail — not autonomous self-modification, which stays permanently out of scope per your constitution.

5. Deployment: local on the Chromebook, and “online”

Local (Crostini): unchanged from your current setup — services run under Crostini, bound to `127.0.0.1` only. Nothing in this blueprint proposes exposing a port to the internet; `MASTER_PLAN_v3.md` explicitly rejected that (`0.0.0.0` binding), and it stays rejected here.

“Online” access, without exposing anything: git is the sync layer, per the runbook §7 — not a tunnel, not a public endpoint.

```
cd ~/jarvis-x
git add -A && git commit -m "checkpoint: $(date -Is)" && git push
```

From any machine — phone, another laptop, a browser via Claude Code’s remote access — you pull the latest commit and continue. This is the correct “online” answer for a \$0-budget, security-conscious setup: your data and models never leave the Chromebook; only code and config sync.

6. Next tasks, in priority order

Income-first per your own standing rule — if this week’s Outlier/Mindrift/Prolific hours aren’t done, they come before anything below.

Tier 1 — resolve open questions (closes every “unverified” item above)

#	Task	Effort	Outcome
1	Run Session 1 of the runbook on the real Chromebook	30 min	Closes §1’s remaining open item permanently
2	Run the JS suite + model-gateway suite on the real machine	30 min	Converts the 6 unmeasured subsystems (vision, 3 voice paths, Kokoro, live data) into knowns
3	Your ruling: Market Intelligence / paper-trading code — delete, or formally re-admit to scope	5 min	Closes §3
4	Your ruling: confirm Kokoro-82M as the sole TTS engine, and what “SILMA-9B + Chatterbox” in the Aug 28 report actually refers to	5 min	Closes §2.4
5	Give test-guard.js and test-shell.js	1.5 h	These are the tests that would have caught the jail-escape

#	Task	Effort	Outcome
	real assertions		bug; right now they can't catch anything

Tier 2 — close real gaps

#	Task	Effort
6	Build the weekly sweep script described in §4.2	2–3 h
7	Real <code>jj status</code> (live provider quota, not a hardcoded string)	2 h
8	Decide and implement JX_NET gating (or make network tests offline-capable)	1.5 h
9	Make the <code>.deb</code> installer actually ship <code>app.py</code> , or drop it in favor of the git-sync deployment in §5	2 h

Tier 3 — only after Tier 1-2

#	Task
10	Electron shell → launchable app (currently 3 files, no build)
11	Single E2E flow test: ask → answer → audio, asserted
12	Quantum track: build the 500-query real dataset from <code>logs/queue.jsonl</code> — the only input that could change §3's quantum verdict, and it improves the classical router either way

7. Source documents and their status

Document	Date	Status in this blueprint
MASTER_PLAN_v3.md	2026-09-03	Canonical current-state source
AS_BUILT.md	2026-09-03	Canonical evidence source
2026-09-repo-triage.md	2026-09	Folded in; archive list already actioned
JARVIS_X_REBUILD_RUNB	2026-09-03	Active process document —

Document	Date	Status in this blueprint
00K.md		its 8-session structure underlies §6
Aug 28 status report	2026-08-28	Folded in for the voice-subsystem fixes; completion % superseded by §2.1
JARVIS_X_v2_Ascension_Plan.pdf	undated (v2.0)	Partially folded in (Universal Model Layer framing); trading, content-engine, and brand/app sections flagged, not adopted — see §3
Prior canonical plan (“Final Plan, Hermes, Chrome OS/Crostini”)	2026-08	Superseded by MASTER_PLAN_v3.md for hardware and completion figures; its scope discipline and cut list remain in force per §3

This document itself should be superseded the moment Tier 1 tasks in §6 produce fresh evidence — treat it the same way AS_BUILT.md treats every document before it.